PlasmaPy: An open source Python package for plasma research and education

Nick Murphy[1], E. T. Everson[2], D. Stańczak-Marikin, D. Schaffner[3], P. V. Heuer[4,5], J. Roberts[5], E. Johnson[6], H. Bagherianlemraski[7], S. Vincena[2], M. Haque[8], and many others

[1] CfA [2] University of California, Los Angeles [3] Bryn Mawr College [4] Laboratory for Laser Energetics [5] University of Rochester [6] University of Delaware [7] MathWorks [8] Columbia University

Abstract

It's like Astropy, but for plasma science! ✨ 🐍

Most astrophysical fluid is plasma!

Figure 1: Coronal mass ejection observed by the Solar Dynamics Observatory at 304 Å on 2011 June 7.

Plasma astrophysics is central to CfA science. Astronomers often refer to any astrophysical fluid as **"gas,"** regardless of the state of matter. As distinct states of matter, **gas** and **plasma** exhibit **qualitatively and quantitatively different behavior on both small and large scales**. **Gas** particles interact via binary collisions, while charged particles in **plasma** exert long-range forces on each other, leading to **collective behavior**. **Gas** dynamics are unaffected by **magnetic fields**, while **magnetic fields** strongly interact with **plasma**. Even a tiny ionization fraction from cosmic rays or photoionization can cause a fluid to behave as a **weakly ionized plasma**.

Growing a software ecosystem

The mission of the PlasmaPy project is to grow an open source software ecosystem for plasma research and education. A software ecosystem is a collection of software projects that are developed together and co-evolve in the same environment. It includes code, documentation, and tests. A software ecosystem includes educational elements, and it includes community. Inspired directly by Astropy and SunPy, PlasmaPy was founded at the CfA — in office P-140.

PlasmaPy is an active participant in the **Python in Heliophysics Community (PyHC)**. PyHC facilitates scientific discovery by growing open source Python software and community for solar and space physics, including through packages like SunPy, SpacePy, pySPEDAS, pysat, and PlasmaPy. In addition to showcasing packages, PyHC's fortnightly meetings often include continuing professional development for research software engineers (RSEs) — activities that would be beneficial to current and future RSEs at the CfA.

PlasmaPy capabilities

PlasmaPy has many useful capabilities for the astrophysical community. `Particle` and `ParticleList` provide object-oriented representations of particles. The `RelativisticBody` class allows straightforward conversion between velocity, the Lorentz factor, energy, and relativistic momentum. PlasmaPy contains a wide variety of functions to calculate plasma parameters, ranging from `Alfven_speed` and `gyroradius` to dielectric tensor elements and Braginskii transport coefficients. PlasmaPy's particle tracker was recently revamped and includes non-relativistic and relativistic Boris particle pushers. PlasmaPy allows the calculation of several types of plasma wave dispersion relations, and well as representations of 1D equilibria. PlasmaPy's synthetic charged particle radiography and Thomson scattering diagnostic analysis capabilities have been widely used to study laser-produced high energy density plasmas.

Table 1: PlasmaPy subpackages, described further at `docs.plasmapy.org`.

Subpackage	Description
`particles`	Representations of particles in plasmas
`formulary`	Formulae for plasma parameters and transport coefficients
`dispersion`	Dispersion relation solvers for plasma waves
`simulation`	Contains a particle tracker with a relativistic Boris pusher
`diagnostics`	For plasma diagnostics such as Langmuir probes, proton radiography, and Thomson scattering
`analysis`	Analysis tools for simulations, experiments, and observations

Examples

PlasmaPy's documentation not only describes the how to use functions and classes, but also the underlying physics — in case you ever need to remember what the `Debye_length` and `upper_hybrid_frequency` signify. The documentation includes numerous example Jupyter notebooks as educational resources. As an executable research poster, the following beginner examples were run using Quarto, Typst, and Jupyter (see `github.com/namurphy/plasmapy-poster`).

```
from plasmapy.particles import Particle, ParticleList
from plasmapy.formulary import plasma_frequency, RelativisticBody
import astropy.units as u
```

```
proton = Particle("p+")
proton.mass  # interoperable with astropy.units
```

$1.6726219 \times 10^{-27}\,\mathrm{kg}$

```
helium_ions = ParticleList(["He-4 1+", "α"])
helium_ions.charge
```

$[1.6021766 \times 10^{-19}, 3.2043533 \times 10^{-19}]\,\mathrm{C}$

```
plasma_frequency(n=0.1*u.cm**-3, particle="e-", to_hz=True)
```

$2839.3025\,\mathrm{Hz}$

```
p = RelativisticBody("proton", total_energy=1e16*u.eV)
print(f"A proton at {p.total_energy.to('PeV')} has β={p.v_over_c} 🙀")
```

```
A proton at 10.0 PeV has β=0.9999999999999956 🙀
```

Laboratory plasma astrophysics diagnostics

Proton radiography involves launching protons through an exploding laser-produced plasma. Protons get deflected by electromagnetic fields before impacting the detector plane. Synthetic radiographs constrain the electromagnetic field configuration.

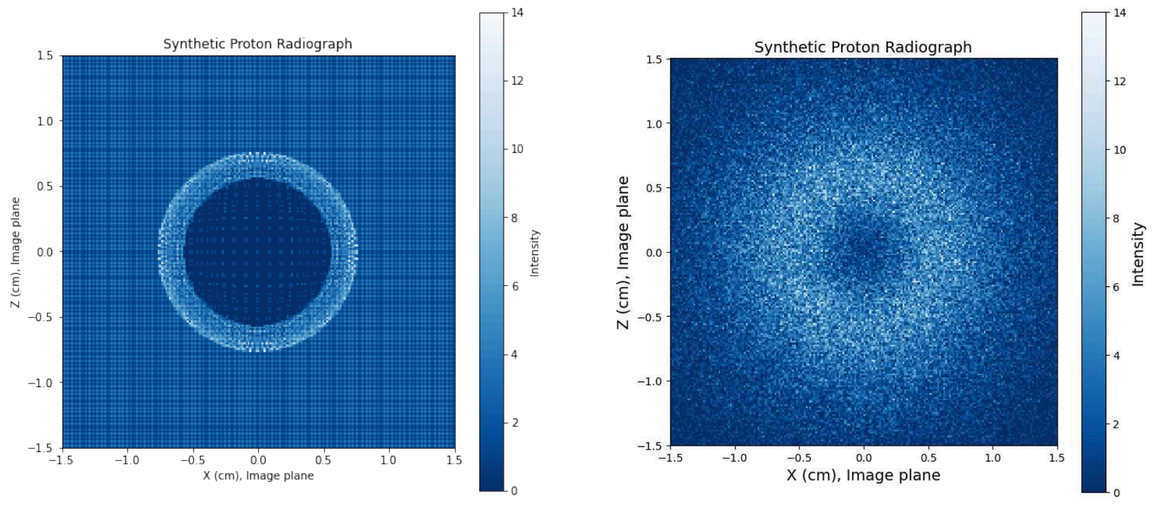

Figure 2: A synthetic proton radiograph of a high energy density plasma (credit: P. Heuer).

Diagnostics like these allow study of effects such as the Biermann battery — a proposed mechanism for creating seed magnetic fields in the early universe that later got amplified through dynamo activity. During one interactive PlasmaPy tutorial, a student managed to create a synthetic proton radiograph that looked like the Death Star (see G. Lucas et al. 1977, 1983).

Future work

To learn more about our proposed development roadmap, we made our most recent NSF Cyberinfrastructure for Sustained Scientific Innovation proposal available on Zenodo at Murphy et al. (2025, doi: 10.1038/150405d0).

Acknowledgments

This work has been supported by NSF CSSI award 1931388 and NASA HTM award 80NSSC25K0360.

Fun fact: I grew up north of Canada!